AIRPORT MANAGEMENT INFORMATION SYSTEM

Database Management Systems and Programming I Term Project

1. Introduction

The purpose of this project is to design and implement a relational database management system for an Airport Management Information System. The system stores, organizes, and manages airport-related information such as airplanes, technicians, employees, tests, and hangars.

The database was designed using MySQL Workbench and implemented with SQL queries. The system allows airport management to store aircraft information, employee information, maintenance test results, and hangar data efficiently.

The project also demonstrates the use of relational database concepts including:

- Primary Keys
- Foreign Keys
- Relationships
- SQL Queries
- Aggregate Functions
- JOIN Operations
- GROUP BY Operations

The designed database simulates a real-life airport management structure and provides statistical information for management purposes.

2. Assumptions

The following assumptions were made while designing the system:

- Each airplane belongs to one plane model.
- Each plane model can belong to multiple airplanes.
- Each technician is also an employee.
- Each employee has a unique social security number (SSN).
- A technician can perform multiple tests.
- Each test event belongs to one airplane, one technician, and one test.
- One hangar can contain multiple airplanes.
- Each airplane has one status at a time.
- Test scores are evaluated over 100 points.

3. ER Diagram

The ER Diagram of the Airport Management Information System was created using MySQL Workbench.

(ER Diagram image should be inserted here.)

4. Relational Schema

Plane_Models

- model_id (PK)
- model_name
- manufacturer
- engine_type

Planes

- plane_id (PK)
- plane_no
- model_id (FK)
- capacity
- manufacture_year
- status

Hangars

- hangar_id (PK)
- hangar_name
- location
- capacity

Employees

- ssn (PK)
- first_name
- last_name
- phone
- salary
- union_membership_no

Technicians

- technician_id (PK)

- ssn (FK)
- specialization_level

Tests

- test_id (PK)
- test_name
- max_score
- duration_hours

Test_Events

- event_id (PK)
- plane_id (FK)
- technician_id (FK)
- test_id (FK)
- test_date
- hours_spent
- score

5. DDL (Data Definition Language)

The following SQL commands were used to create the database tables:

```
CREATE TABLE Plane_Models (
   model_id INT PRIMARY KEY AUTO_INCREMENT,
   model_name VARCHAR(100) NOT NULL,
   manufacturer VARCHAR(100),
   engine_type VARCHAR(50)
);

CREATE TABLE Planes (
   plane_id INT PRIMARY KEY AUTO_INCREMENT,
   plane_no VARCHAR(20) UNIQUE,
   model_id INT,
   capacity INT,
   manufacture_year YEAR,
   status VARCHAR(50),

   FOREIGN KEY (model_id)
   REFERENCES Plane_Models(model_id)
);
```

```
CREATE TABLE Employees (
   ssn VARCHAR(20) PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   phone VARCHAR(20),
   salary DECIMAL(10,2),
   union_membership_no VARCHAR(50)
);
```

6. DML (Data Manipulation Language)

Sample data inserted into the database:

```
INSERT INTO Plane_Models
(model_name, manufacturer, engine_type)
VALUES
('Boeing 737', 'AJet', 'Jet'),
('Airbus A320', 'Pegasus', 'Jet'),
('Boeing 777', 'Turkish Airlines', 'Jet');
```

```
INSERT INTO Planes
(plane_no, model_id, capacity, manufacture_year, status)
VALUES
('AJT1001', 1, 180, 2018, 'Active'),
('PGS2001', 2, 220, 2020, 'Maintenance'),
('THY3001', 3, 350, 2022, 'Active');
```

7. SQL Queries

Query 1 – Display all airplanes

```
SELECT * FROM Planes;
```

Query 2 – Highest test score

```
SELECT MAX(score) AS highest_score
FROM Test_Events;
```

Query 3 – Average test score

```
SELECT AVG(score) AS average_score
FROM Test_Events;
```

Query 4 – Number of tests performed by technicians

```
SELECT technician_id, COUNT(*) AS total_tests
FROM Test_Events
GROUP BY technician_id;
```

Query 5 – Airplane with highest capacity

```
SELECT *
FROM Planes
ORDER BY capacity DESC
LIMIT 1;
```

Query 6 – Maintenance airplanes

```
SELECT *
FROM Planes
WHERE status = 'Maintenance';
```

Query 7 – Test scores above 80

```
SELECT *
FROM Test_Events
WHERE score > 80;
```

Query 8 – Average employee salary

```
SELECT AVG(salary) AS average_salary
FROM Employees;
```

Query 9 – Highest paid employee

```
SELECT *
FROM Employees
ORDER BY salary DESC
LIMIT 1;
```

Query 10 – Airplanes and their tests

```
SELECT
Planes.plane_no,
Tests.test_name,
Test_Events.score
FROM Test_Events
JOIN Planes
ON Test_Events.plane_id = Planes.plane_id
JOIN Tests
ON Test_Events.test_id = Tests.test_id;
```

8. Conclusion

In this project, a relational database system for airport management was successfully designed and implemented using MySQL.

The project includes multiple related tables, foreign key relationships, SQL queries, and statistical reports. The database structure provides an efficient way to manage airport operations including aircraft management, employee tracking, technician activities, and aircraft testing processes.

This project helped improve practical knowledge about relational database systems, SQL programming, and database design principles.